TEK-UP UNIVERSITY

AI ENGINEERING

CAHIER DES CHARGES

OPSPILOT

Evidence-Quality-Aware Kubernetes Incident Diagnosis

Prototype de recherche et d'ingénierie IA pour le diagnostic d'incidents Kubernetes fondé sur la qualité des preuves

Élément	Information
Matière	AI Engineering
Établissement	TEK-UP University - Tunisie
Étudiants	Bilel Amri et Mohamed Amine Gotai
Encadrant	À compléter
Version du document	V1.3 - Version consolidée pour validation pédagogique
Date	2 novembre 2026

Document de spécification - Les fonctionnalités et résultats décrits sont des objectifs à réaliser et à vérifier.

Gestion du document

Champ	Valeur
Titre	OpsPilot - Evidence-Quality-Aware Kubernetes Incident Diagnosis
Version	1.3
Statut	Proposition technique soumise à validation de l'enseignant
Auteurs	Bilel Amri ; Mohamed Amine Gotai
Date de référence	2 novembre 2026
Public cible	Enseignant AI Engineering, évaluateurs, membres de l'équipe et futurs contributeurs
Périmètre	Prototype local reproductible ; aucune remédiation Kubernetes automatique

Résumé exécutif

OpsPilot est un prototype d'ingénierie IA qui étudie la fiabilité du diagnostic d'incidents Kubernetes lorsque les observations opérationnelles sont incomplètes, anciennes, contradictoires ou insuffisamment traçables. Le système collecte des informations en lecture seule depuis un cluster Kubernetes contrôlé, normalise ces observations sous forme de preuves structurées, puis applique un Evidence Quality Gate (EQG) déterministe avant de permettre à un modèle de langage de formuler un diagnostic.

Le système combine quatre éléments complémentaires : (1) un serveur MCP exposant des outils Kubernetes en lecture seule ; (2) une base documentaire RAG fondée sur la documentation officielle Kubernetes, versionnée et citée ; (3) un EQG qui contrôle la complétude, la fraîcheur, la cohérence et la provenance des observations ; et (4) un workflow d'investigation utilisant LangChain et LangGraph, avec validation déterministe du rapport généré. Une interface web React/TypeScript permet de consulter les incidents, les preuves et les résultats d'évaluation.

La contribution principale visée est l'implémentation et l'évaluation d'un mécanisme de contrôle de qualité des preuves spécifique aux observations Kubernetes. L'étude mesurera son effet sur le taux de diagnostics non étayés, la capacité d'abstention et la couverture diagnostique, sans supposer à l'avance que l'EQG améliore les résultats.

Décision de périmètre

Le MVP reste volontairement limité : un cluster Kind local, trois familles initiales d'incidents, quatre conditions de qualité des preuves, quatre outils MCP en lecture seule, un pipeline RAG et une interface web. Les extensions à six scénarios sont conditionnelles à la validation du MVP.

Sommaire

- 1. Contexte, problématique et justification
- 2. État de l'art et positionnement
- 3. Objectifs et question de recherche
- 4. Périmètre, hypothèses et contraintes
- 5. Parties prenantes et cas d'utilisation
- 6. Exigences fonctionnelles et non fonctionnelles
- 7. Architecture cible et flux de données
- 8. Gestion documentaire et pipeline RAG
- 9. Collecte opérationnelle et serveur MCP
- 10. Evidence Quality Gate (EQG)
- 11. Orchestration LLM et génération du rapport
- 12. Protocole expérimental et benchmarking
- 13. Interface utilisateur et API

- 14. Modèle de données
- 15. Sécurité, éthique et limites
- 16. Tests, qualité logicielle et MLOps
- 17. Organisation de l'équipe et planning
- 18. Livrables et critères d'acceptation
- 19. Risques et plans de mitigation
- 20. Faisabilité et ressources
- 21. Glossaire
- 22. Références

1. Contexte, problématique et justification

1.1 Contexte métier

Les applications cloud-native sont exécutées sur des clusters Kubernetes et génèrent plusieurs types d'observations : états de Pods et de conteneurs, journaux applicatifs, événements Kubernetes et configuration déclarative. Lorsqu'un incident survient, l'ingénieur doit corréler ces informations avec la documentation technique, la version du cluster et les procédures de dépannage. Cette investigation mobilise des sources distinctes et peut être ralentie par des preuves manquantes, anciennes ou mal associées à la ressource concernée.

Les assistants fondés sur des LLM peuvent synthétiser des observations et de la documentation, mais une réponse fluide ne garantit ni que les observations appartiennent au bon Pod ou conteneur, ni qu'elles sont assez récentes, ni que les conclusions sont soutenues par les éléments cités. Le projet se concentre sur ce risque de fiabilité.

1.2 Problématique

Comment limiter les diagnostics de cause racine non étayés lorsqu'un LLM reçoit des observations Kubernetes incomplètes, périmées ou contradictoires, tout en conservant une couverture diagnostique utile ?

1.3 Conséquences d'une preuve de mauvaise qualité

- Une log précédente peut être interprétée comme le journal courant si son contexte de collecte n'est pas conservé.
- Un événement lié à un ancien UID de Pod peut être associé à tort à un nouveau Pod portant le même nom.
- Un statut de Pod et un état de conteneur peuvent être confondus ; ils décrivent des niveaux différents du cycle de vie.
- Une documentation correcte mais incompatible avec la version du cluster peut conduire à une recommandation inadaptée.
- Une réponse du LLM peut citer une source qui ne soutient pas réellement le fait affirmé.

1.4 Pourquoi un Evidence Quality Gate ?

L'EQG est une couche logicielle déterministe qui vérifie des propriétés explicites des observations avant la génération du diagnostic. Il ne prétend pas vérifier la vérité absolue d'un capteur ou d'un journal. Il vérifie des conditions techniques contrôlables : présence des champs requis par scénario, cohérence d'identité et de temporalité, traçabilité de la source et respect d'une politique de fraîcheur définie par type d'observation.

Principe de prudence

L'EQG ne remplace ni le jugement d'un ingénieur ni la validation de la cause racine. Il détermine si les observations disponibles sont suffisamment structurées et cohérentes pour autoriser un diagnostic confirmé, un diagnostic prudent ou une abstention.

1.5 Résultat attendu

Un prototype local capable d'investiguer un incident reproductible, d'expliquer quelles preuves ont été collectées, de signaler leurs limites, de récupérer la documentation compatible et de produire un rapport vérifiable. L'évaluation doit permettre d'identifier non seulement les succès, mais également les cas où le gate rejette à tort de bonnes preuves ou laisse passer des preuves dégradées.

2. État de l'art et positionnement

Le diagnostic d'incidents Kubernetes assisté par IA est déjà un domaine actif. OpsPilot ne revendique ni l'invention de l'AIOps, ni la première intégration MCP avec Kubernetes, ni le premier système d'abstention. Le positionnement porte sur l'implémentation et l'évaluation d'un gate déterministe ciblant la qualité des observations opérationnelles Kubernetes.

Référence / solution	Apport identifié	Conséquence pour OpsPilot
K8sGPT [R14]	Analyse de ressources Kubernetes, explications IA et serveur MCP exposant des outils de diagnostic.	Ne pas revendiquer le diagnostic Kubernetes via LLM ou MCP comme nouveauté.
HolmesGPT [R15]	Outils de diagnostic Kubernetes en lecture seule et intégrations MCP / observabilité.	Limiter la comparaison au périmètre documenté ; éviter la course aux fonctionnalités.
Botkube [R16]	Assistant et intégrations pour événements, logs et opérations Kubernetes.	Le monitoring et l'inspection d'événements sont des capacités existantes.
ARGUS [R17]	RCA Kubernetes fondée sur MCP et données d'observabilité ; étude de scénarios de fault injection.	MCP et corrélation d'observabilité ne constituent pas la contribution centrale.
Cloud-OpsBench [R18]	Infrastructure d'évaluation reproductible pour RCA cloud ; snapshots et nombreux cas de fautes.	Ne pas prétendre inventer les snapshots reproductibles ou le benchmark RCA.
TriFleetRCA [R19]	RCA Kubernetes locale, sélection de preuves, garde-fou de runbooks et qualité de citation.	Évaluer séparément exactitude du diagnostic et attribution correcte des preuves.
R2Act [R20]	Évaluation de la validité des actions de récupération après diagnostic.	Justifie de séparer diagnostic et remédiation ; dataset non requis pour le MVP.
K8sGPT Calibration & Abstention Benchmark [R21]	Benchmark communautaire autour de calibration, abstention et risque d'action.	L'abstention seule n'est pas une revendication d'originalité.

2.1 Positionnement différenciant visé

OpsPilot propose d'étudier un composant EQG explicite, déterministe et testable, qui normalise des métadonnées de provenance et applique des règles de complétude, fraîcheur et cohérence adaptées à chaque scénario. L'étude compare ensuite les sorties d'un LLM sans gate, avec RAG, puis avec RAG et EQG.

2.2 Limites de l'état de l'art

Les publications et outils cités couvrent des problèmes proches mais leurs périmètres, protocoles et données diffèrent. La présente spécification ne prétend pas avoir effectué une revue systématique exhaustive. Une matrice de littérature plus complète sera construite pendant S0, avec lecture des articles complets, vérification du statut de publication, des licences et de la disponibilité réelle des dépôts ou datasets.

Formulation autorisée

La contribution sera formulée comme une évaluation expérimentale d'un mécanisme Kubernetes-spécifique dans un périmètre contrôlé. Aucun énoncé du type « premier système au monde » ou « supérieur aux solutions existantes » ne sera utilisé sans preuve.

3. Objectifs et question de recherche

3.1 Objectif général

Concevoir, implémenter et évaluer un prototype d'investigation d'incidents Kubernetes qui vérifie la qualité des preuves opérationnelles avant de laisser un LLM présenter une cause racine comme confirmée, tout en intégrant les exigences d'AI Engineering du cours.

3.2 Question de recherche principale

RQ

Un Evidence Quality Gate déterministe, spécifique aux observations Kubernetes, peut-il réduire le taux de diagnostics de cause racine non étayés lorsque les preuves opérationnelles sont manquantes, périmées ou contradictoires, tout en maintenant une couverture diagnostique utile ?

3.3 Objectifs spécifiques

1. Construire un laboratoire Kubernetes local et reproductible pour générer des incidents contrôlés.
2. Collecter les observations opérationnelles en lecture seule au moyen d'un serveur MCP.
3. Définir un modèle de preuve structuré avec identité, provenance, horodatage et lien à un snapshot.
4. Implémenter un EQG déterministe avec des règles explicites par scénario.
5. Construire un corpus documentaire Kubernetes versionné et un pipeline RAG avec citations.
6. Mesurer séparément la qualité du retrieval, la qualité de génération et la fiabilité du diagnostic.
7. Développer une interface web pour consulter incidents, preuves, décisions EQG et résultats expérimentaux.
8. Documenter les limites, erreurs et compromis du système.

3.4 Hypothèses de recherche

ID	Hypothèse	Mesure associée
H1	L'EQG détecte une partie substantielle des snapshots volontairement dégradés selon les règles définies.	Faux-pass, faux-bloc et matrice de confusion EQG.
H2	L'ajout de l'EQG réduit le taux de diagnostics confirmés non étayés par rapport au baseline RAG + LLM.	Unsupported Diagnosis Rate.
H3	La réduction des diagnostics non étayés peut être obtenue sans réduire excessivement la couverture.	Diagnostic Coverage et couverture sélective.
H4	Le filtrage documentaire par version améliore la pertinence des passages retournés lorsque la question est réellement sensible à la version.	Recall@k, MRR, nDCG et exactitude des citations.
H5	Une stratégie hybride et/ou un reranker peut améliorer le classement des passages pertinents par rapport à une seule stratégie de recherche.	Recall@k, MRR, nDCG, latence.

Les hypothèses sont des propositions à tester. Aucun résultat n'est présumé.

3.5 Contributions attendues

Contribution	Description	Nature
Technique	EQG déterministe : règles de complétude, fraîcheur, cohérence et provenance.	Implémentation logicielle

Contribution	Description	Nature
Données	Snapshots d'incidents et variantes contrôlées : complet, manquant, périmé, contradictoire.	Artefacts expérimentaux étiquetés
Évaluation	Comparaisons RAG / non-RAG / EQG, plus benchmark autonome de retrieval et génération.	Protocole et résultats reproductibles

4. Périmètre, hypothèses et contraintes

4.1 Inclus dans le MVP

- Un cluster local Kind, dans un environnement isolé et jetable.
- Une application de démonstration légère, déployée dans un namespace dédié.
- Trois familles initiales d'incidents : CrashLoopBackOff, ImagePullBackOff et OOMKilled.
- Quatre conditions de qualité : complète, manquante, périmée et contradictoire.
- Quatre outils MCP en lecture seule : list_pods, get_pod_status, get_pod_logs et get_recent_events.
- Un EQG Python déterministe et une validation déterministe après génération.
- Un corpus de documentation officielle Kubernetes, versionné sur deux versions retenues.
- Un pipeline RAG avec recherche lexicale PostgreSQL, recherche dense, recherche hybride et expérimentation de reranking.
- Un workflow d'investigation utilisant LangChain et LangGraph, sans architecture multi-agents complexe.
- Une interface web React/TypeScript et un backend FastAPI.
- Un benchmark RAG d'au moins 50 questions, avec jugements de pertinence et séparation développement / holdout.
- Un jeu d'évaluation EQG initial de 12 conditions (3 scénarios x 4 conditions), extensible à 24 conditions.

4.2 Hors périmètre du MVP

- Toute remédiation automatique ou mutation du cluster Kubernetes (delete, restart, patch, scale, drain, apply).
- La lecture de Secrets Kubernetes ou de données personnelles/production.
- Un déploiement multi-cloud ou multi-cluster de production.
- Une plateforme AIOps concurrente complète de K8sGPT, HolmesGPT ou Botkube.
- La reproduction du benchmark Cloud-OpsBench ou du dataset R2Act.
- Le fine-tuning ou l'entraînement d'un modèle, sauf accord pédagogique et temps disponible.
- Une architecture distribuée en microservices ; un monolithe modulaire suffit.
- La promesse d'une amélioration statistiquement généralisable à tous les incidents Kubernetes.

4.3 Extensions conditionnelles

Si le MVP est stable et si le temps et le matériel le permettent, l'équipe pourra ajouter Pending Pod, ConfigMap manquant et échec de liveness probe. Un échec readiness ne doit pas être présenté comme équivalent à un échec liveness : readiness retire normalement le Pod du trafic de Service, alors que liveness peut provoquer le redémarrage du conteneur.

4.4 Hypothèses de travail

Hypothèse	Conséquence / vérification
Docker Engine disponible sur le poste Windows	Vérifier Docker Desktop, WSL2, mémoire et espace disque avant le cluster.
Kind et kubectl installables	Figer les versions dans les instructions et la CI après test local.

Hypothèse	Conséquence / vérification
Python >= 3.11	Compatible avec la ligne stable actuelle du SDK MCP Python v2 [R9].
Modèle compatible JSON structuré	Valider le provider / modèle sur un test d'appel avant intégration.
Corpus public réutilisable	Respecter CC BY 4.0 pour le dépôt Kubernetes et attribuer les sources [R7].
Équipe de deux étudiants	Répartition primaire définie en section 17 et validée par les deux membres.

4.5 Contraintes de ressources

Le cœur du système doit être exécutable localement sans abonnement payant obligatoire. L'inférence locale est privilégiée si le matériel est suffisant ; sinon, un fournisseur externe peut être utilisé uniquement s'il est autorisé, avec secrets hors dépôt et coût documenté. Le projet ne doit pas dépendre d'un GPU dédié. Le dimensionnement final sera décidé après contrôle RAM/CPU/disque et essai de charge du laboratoire.

5. Parties prenantes et cas d'utilisation

Acteur	Rôle	Besoins
Ingénieur DevOps / SRE	Utilisateur principal	Comprendre un incident, voir les preuves, connaître les limites du diagnostic.
Développeur backend	Utilisateur secondaire	Corréler les logs de l'application, les états du conteneur et la documentation.
Étudiants	Concepteurs / opérateurs du prototype	Créer les scénarios, maintenir le corpus, exécuter les expériences.
Enseignant	Évaluateur	Vérifier les cinq exigences du cours, le code, les commits, le rapport et la démonstration.

5.1 Cas d'utilisation principaux

ID	Cas	Précondition	Résultat attendu
UC-01	Déclarer un incident	Utilisateur authentifié / cluster de test accessible.	Un incident est enregistré avec namespace, sélection de ressource et fenêtre temporelle.
UC-02	Collecter les preuves	Accès MCP en lecture seule.	Les outils collectent les états, logs et événements avec métadonnées de provenance.
UC-03	Évaluer la qualité des preuves	Snapshot normalisé.	EQG fournit indicateurs, anomalies, décision et raisons.
UC-04	Interroger la documentation	Corpus indexé et version connu ou explicitement inconnu.	Passages pertinents, version, section et URL sont retournés.
UC-05	Générer un diagnostic	État EQG et contexte RAG disponibles.	Rapport structuré avec hypothèses, preuves E, citations D et incertitude.
UC-06	Consulter une évaluation	Résultat d'expérience stocké.	L'utilisateur compare les métriques et les configurations.
UC-07	Exporter un rapport	Investigation terminée.	Rapport JSON/Markdown contenant la traçabilité et les limites.

5.2 Scénario nominal

- L'utilisateur choisit un incident et le namespace autorisé.
- Le backend lance les outils MCP requis et conserve les résultats bruts.
- Les observations sont normalisées et liées à un snapshot immuable.
- L'EQG calcule les indicateurs et produit une décision avec les règles déclenchées.
- Le RAG récupère la documentation compatible avec la version Kubernetes sélectionnée.
- Le LLM produit un rapport structuré respectant les contraintes EQG.
- Un validateur vérifie les références E et D, les identifiants et le niveau de certitude autorisé.
- Le rapport, le snapshot, les appels d'outils et les résultats sont enregistrés.

6. Exigences fonctionnelles et non fonctionnelles

6.1 Exigences fonctionnelles

ID	Exigence	Module / preuve de test	Critère de vérification
FR-01	Créer et consulter un incident de test.	API incident ; test API.	Démonstration de création / historique.
FR-02	Collecter la liste des Pods du namespace autorisé.	MCP list_pods ; test intégration.	Résultat réel issu du cluster.
FR-03	Collecter le détail Pod, états de conteneurs et ressources déclarées.	MCP get_pod_status ; test de schéma.	Phase et états distingués.
FR-04	Récupérer logs courants ou précédents.	MCP get_pod_logs(previous).	Logs précédents disponibles lorsque Kubernetes les conserve.
FR-05	Récupérer les événements liés à une ressource.	MCP get_recent_events.	Événements associés au bon objet et à sa fenêtre.
FR-06	Normaliser chaque observation en preuve traçable.	Evidence schema ; tests JSON Schema.	Chaque preuve a source, identité, temps de collecte et snapshot.
FR-07	Évaluer complétude, fraîcheur, cohérence et provenance.	EQG déterministe.	Décision explicable et reproductible.
FR-08	Bloquer un diagnostic confirmé non autorisé par EQG.	Policy enforcement + validator.	Test négatif de sortie LLM.
FR-09	Ingestérer documents et métadonnées versionnées.	RAG ingestion.	Corpus indexé et manifesté.
FR-10	Récupérer et classer les passages pertinents.	Lexical, dense, hybrid, reranker.	Résultats mesurables via qrels.
FR-11	Afficher les citations documentaires et les preuves opérationnelles séparément.	Report schema + UI.	Références Dxxx et Exxx distinctes.
FR-12	Exécuter le workflow LangGraph.	Graph nodes / state.	Traçage des étapes et gestion des erreurs.
FR-13	Enregistrer les snapshots, décisions, résultats et métriques.	PostgreSQL.	Historique consultable.
FR-14	Afficher les incidents et preuves dans l'interface web.	React + API.	Parcours complet démontrable.
FR-15	Exécuter les évaluations RAG, EQG et diagnostic.	Evaluation runner.	Rapports reproductibles.

6.2 Exigences non fonctionnelles

ID	Exigence
NFR-01 Fiabilité	Toute erreur de collecte est explicitement signalée ; aucune absence de donnée n'est transformée en donnée neutre.
NFR-02 Reproductibilité	Version des outils, modèle, prompt, corpus, paramètres et snapshots enregistrés pour chaque expérience.
NFR-03 Sécurité	RBAC restrictif, absence de droit de mutation, aucun accès Secret dans le MVP.
NFR-04 Traçabilité	Chaque résultat permet de retrouver l'incident, le snapshot, les outils, les preuves et les documents cités.

ID	Exigence
NFR-05 Maintenabilité	Code modulaire, types explicites, erreurs documentées et tests automatisés.
NFR-06 Portabilité	Exécution locale via instructions vérifiées et configurations versionnées.
NFR-07 Performance	Latence mesurée par étape (collecte, gate, retrieval, génération) ; aucun SLA de production revendiqué.
NFR-08 Confidentialité	Aucune donnée de cluster de production ; suppression / masquage des valeurs sensibles du laboratoire.
NFR-09 Accessibilité	Interface lisible, états loading/error/empty et navigation cohérente.

6.3 Exigences de sécurité

- SEC-01 : les identifiants Kubernetes sont limités à un ServiceAccount dédié au namespace de laboratoire.
- SEC-02 : les outils MCP n'exposent que get/list sur les ressources autorisées et l'accès pods/log nécessaire ; aucune permission create/update/patch/delete.
- SEC-03 : aucun secret Kubernetes n'est collecté ni transmis au modèle.
- SEC-04 : les logs et documents sont traités comme données non fiables, jamais comme instructions de contrôle.
- SEC-05 : toute entrée de tool est validée par schéma, bornes, namespace autorisé et timeout.
- SEC-06 : les sorties du modèle sont validées ; les identifiants de preuve et de document doivent exister.
- SEC-07 : les clés API et tokens sont conservés hors du dépôt, dans des variables d'environnement.
- SEC-08 : le MVP ne possède aucune fonction de remédiation Kubernetes.

7. Architecture cible et flux de données

L'architecture est un monolithe modulaire : un backend FastAPI regroupe l'API, la coordination du workflow, l'accès RAG et le service EQG. Le serveur MCP constitue un composant séparé pour isoler les droits et le protocole d'accès au cluster. PostgreSQL stocke les métadonnées, snapshots et résultats ; pgvector est utilisé uniquement pour la recherche dense. React/TypeScript fournit l'interface web.

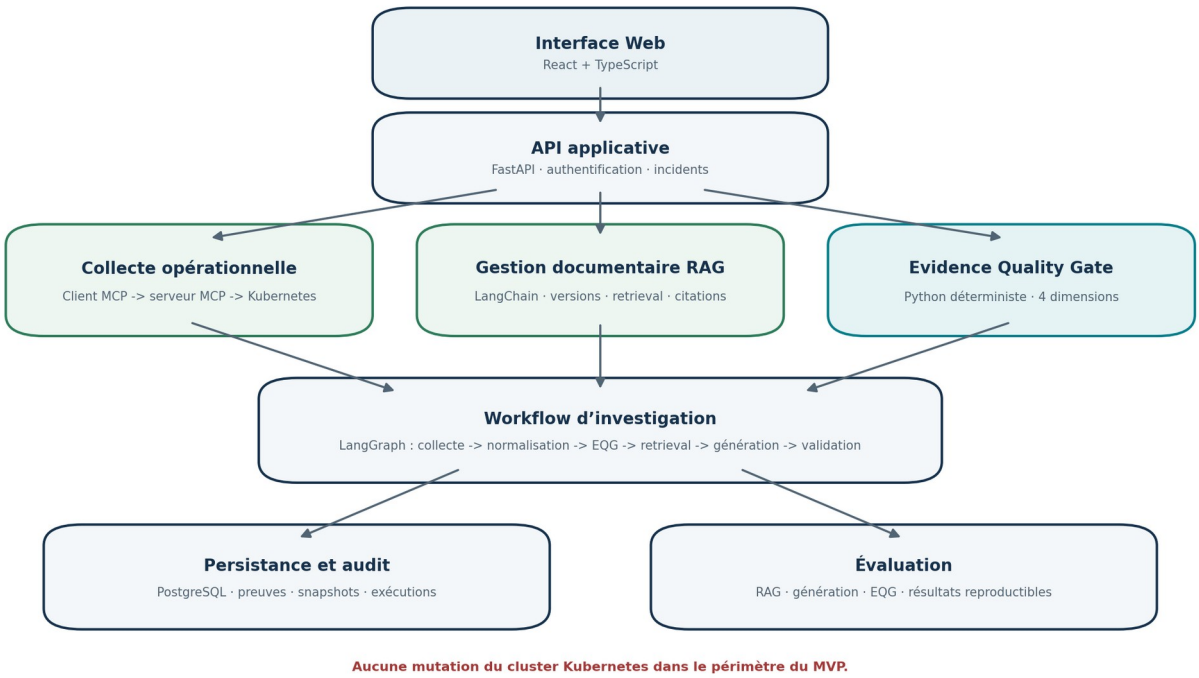


Figure 1 - Architecture logique cible d'OpsPilot (proposition).

7.1 Responsabilités par couche

Couche	Responsabilité	Technologies proposées
Interface	Déclarer incident, consulter preuve, EQG, diagnostic et métriques.	React, TypeScript
API	Validation des requêtes, contrôle d'accès, orchestration, persistance.	FastAPI, Pydantic
Collecte	Accéder aux ressources Kubernetes et retourner des preuves structurées.	MCP Python SDK, Kubernetes Python client ou API
EQG	Évaluer les propriétés des observations avec des règles déterministes.	Python pur, schémas typés
RAG	Ingestion, métadonnées, retrieval, reranking et citations.	LangChain, PostgreSQL FTS, pgvector
Workflow	Gérer les étapes, erreurs, état et décisions de l'investigation.	LangGraph
Données	Persister incidents, snapshots, documents, résultats et audits.	PostgreSQL, pgvector optionnel
Qualité	Tests, CI, rapport d'évaluation et suivi des expériences.	pytest, GitHub Actions, MLflow optionnel

7.2 Choix d'architecture

- Monolithe modulaire plutôt que microservices : plus simple à développer, tester et déployer pour une équipe de deux étudiants.
- EQG sous forme de module Python déterministe, indépendant du fournisseur LLM.
- MCP séparé du backend applicatif afin de réduire le périmètre des permissions Kubernetes.
- PostgreSQL comme stockage principal ; pgvector seulement si la recherche dense est retenue.
- LangChain pour les composants RAG ; LangGraph pour un workflow contrôlé, sans multiplier les agents.

7.3 Flux de données global

Incident utilisateur
-> FastAPI valide le namespace et l'identifiant de ressource
-> client MCP appelle les outils de lecture
-> réponses brutes conservées et normalisées
-> snapshot immuable créé
-> EQG évalue la qualité des preuves
-> RAG récupère la documentation de la version ciblée
-> LLM produit un rapport structuré selon la décision EQG
-> validateur contrôle claims, références E/D et statut autorisé
-> PostgreSQL conserve le résultat et l'audit

8. Gestion documentaire et pipeline RAG

8.1 Corpus de référence

Le corpus principal est la documentation officielle Kubernetes. À la date de référence du document, Kubernetes maintient les branches mineures 1.37, 1.36 et 1.35 ; le site fournit la documentation de la version courante et des versions précédentes [R1, R2]. Les versions 1.36 et 1.37 sont donc retenues pour l'étude initiale. Les versions exactes et les pages collectées seront figées dans un manifest de corpus.

Le dépôt officiel kubernetes/website indique une licence CC-BY-4.0 [R3]. Le dépôt OpsPilot doit conserver l'attribution, l'URL d'origine, la version, la date de collecte et le texte de licence. Les postmortems publics peuvent être utilisés comme références bibliographiques ou liens externes ; leur redistribution dans le dépôt sera exclue sauf licence clairement vérifiée.

Thème documentaire	Exemples de sujets	Utilité
Cycle de vie Pod / conteneur	phases, Waiting, Terminated, restart policy	Interpréter les états sans les confondre.
Dépannage Pod	describe, logs, logs --previous, événements	Dépannage CrashLoopBackOff.
Images et démarrage	pull policy, image reference, registry	Dépannage ImagePullBackOff.
Ressources	requests, limits, mémoire, OOM	Dépannage OOMKilled.
Configuration	ConfigMap, variables d'environnement, volumes	Dépannage configuration.
Probes	liveness, readiness, startup	Distinguer restart et retrait du trafic.
RBAC et API	Roles, subresources, Event API	Concevoir une collecte sécurisée.

8.2 Taille initiale du corpus

Cible de départ : 30 à 50 pages pertinentes par version, soit environ 60 à 100 pages avant déduplication. Cette cible est indicative ; le corpus final sera défini par un manifest listant chaque page et son statut de disponibilité. Le projet ne dépend pas de milliers de documents.

8.3 Pipeline d'ingestion

9. Découverte : sélectionner les pages pertinentes à partir des sources officielles et du plan de corpus.
10. Collecte : télécharger ou extraire le contenu dans le format prévu, en conservant l'URL source.
11. Normalisation : nettoyer navigation, menus et contenu répétitif sans supprimer les détails techniques nécessaires.
12. Segmentation : découper en sections ou chunks cohérents, conserver le titre, les ancres et la hiérarchie.
13. Métadonnées : attacher version, composant, catégorie, langue, type de document, URL, licence et date de collecte.
14. Déduplication : détecter les contenus identiques ou quasi-identiques entre versions et préserver la provenance.
15. Indexation : stocker les chunks et champs lexicaux ; calculer les embeddings pour la recherche dense.
16. Mise à jour : détecter les changements et réindexer uniquement les pages modifiées, en conservant les versions du corpus.

8.4 Modèle de métadonnées documentaire

Champ	Type / exemple	Règle
document_id	DOC-0001	Identifiant stable du document source.
chunk_id	CH-0001	Identifiant unique du passage.
title / section	Debugging Pods / Examining logs	Titre lisible et section exacte.
source_url	URL officielle versionnée	Lien vers la page source, non reconstruit artificiellement.
k8s_version	1.36 ou 1.37	Version explicite ; unknown interdit pour filtre strict.
component / category	Pod / Troubleshooting	Taxonomie contrôlée.
document_type	official_documentation	Type de source.
language	en	Langue du contenu.
license	CC-BY-4.0	Licence et attribution.
collected_at	ISO-8601 UTC	Horodatage de collecte du corpus.
content_hash	SHA-256	Détection de changement.

8.5 Stratégies de recherche comparées

Configuration	Description	Précaution de terminologie
R1 - Recherche lexicale PostgreSQL	tsvector + tsquery + ts_rank ou ts_rank_cd.	Ne pas appeler cette configuration BM25.

Configuration	Description	Précaution de terminologie
R2 - Recherche dense	Embeddings + similarité vectorielle via pgvector.	Modèle d'embedding, dimensions et normalisation figés.
R3 - Recherche hybride	Combinaison lexical + dense, par exemple via Reciprocal Rank Fusion.	Documenter la stratégie de fusion et ses paramètres.
R4 - Hybride + reranker	Reranking des candidats avec cross-encoder.	Mesurer gain de ranking et surcoût de latence.

8.6 Citations

Les citations documentaires doivent identifier au minimum le chunk, le document, la section, la version et l'URL source. Le rapport ne doit pas utiliser une citation documentaire comme preuve directe d'un événement dans le cluster. Les observations opérationnelles sont citées séparément au moyen d'identifiants E.

Distinction obligatoire

E001, E002… désignent des observations Kubernetes collectées. D001, D002… désignent des passages de documentation. Une cause racine doit être reliée aux observations E ; les passages D expliquent le mécanisme ou la procédure, mais ne prouvent pas que cet événement s'est produit.

9. Collecte opérationnelle et serveur MCP

9.1 Rôle du MCP

Le Model Context Protocol standardise la découverte et l'invocation d'outils par un client. OpsPilot utilise un serveur MCP dédié pour exposer des fonctions de lecture de l'état Kubernetes. Un appel MCP réel depuis le workflow est requis : la simple présence du serveur ou d'un fichier de schéma ne suffit pas à satisfaire l'exigence de Tool Calling / MCP [R10].

9.2 Outils du MVP

Outil	Paramètres clés	Données retournées	Permissions minimales
list_pods	namespace autorisé, label_selector optionnel	Nom, UID, phase, node, labels filtrés, noms des conteneurs, timestamps pertinents.	pods: get/list
get_pod_status	namespace, pod_name	Pod phase ; pour chaque conteneur : state, waiting/terminated reason, restartCount, lastState ; pour spec : image, requests/limits.	pods: get
get_pod_logs	namespace, pod_name, container_name, previous=false, tail_lines borné	Texte logs, indicateur current/previous, contexte du Pod/conteneur, collect time.	pods/log: get
get_recent_events	namespace, involved_object_kind/name/uid, since optionnel, limit borné	reason, type, message, involvedObject, eventTime/first/last time quand disponibles, count.	events: get/list

9.3 Contrats de sortie et distinction Kubernetes

Le modèle normalisé doit distinguer le Pod phase de l'état de chaque conteneur. Les phases Pod sont Pending, Running, Succeeded, Failed et Unknown. Les états de conteneur sont Waiting, Running et Terminated. CrashLoopBackOff est généralement un reason associé à un état Waiting et non une phase Pod. Un Pod peut afficher phase Running alors qu'un conteneur est en train de redémarrer [R4].

```
{
 "pod": {
  "name": "demo-api",
  "uid": "<pod-uid>",
  "namespace": "opspilot-lab",
  "phase": "Running",
  "resource_version": "<opaque-api-version>"
 },
 "containers": [{
  "name": "api",
  "state": {"waiting": {"reason": "CrashLoopBackOff"}},
  "last_state": {"terminated": {"reason": "Error", "exit_code": 1}},
  "restart_count": 3,
  "spec": {
   "image": "opspilot-demo:dev",
   "resources": {"requests": {"memory": "64Mi"},
                 "limits": {"memory": "128Mi"}}
  }
 }]
}
```

Exemple illustratif de contrat. Les valeurs ne sont pas des résultats de laboratoire.

9.4 Sécurité et RBAC

- Créer un ServiceAccount dédié à la collecte OpsPilot dans le namespace de laboratoire.
- Autoriser uniquement get/list sur pods et events, et get sur pods/log selon les besoins exacts.
- Ne pas autoriser secrets, exec, attach, port-forward, create, update, patch, delete ou proxy.
- Tester les refus avec kubectl auth can-i pour chaque verbe et ressource interdits.
- Restreindre les namespaces et limiter la taille des logs et événements retournés.
- Le processus de collecte doit définir des timeouts et traiter les erreurs API sans inventer de résultats.

La documentation RBAC Kubernetes précise que l'accès aux logs d'un Pod correspond au sous-resource pods/log ; les droits sur pods n'impliquent pas implicitement la permission sur les logs [R11].

9.5 Version SDK

Le SDK Python officiel MCP documente actuellement la ligne v2 stable et requiert Python 3.11 ou supérieur. La classe serveur et les imports ont évolué entre v1 et v2 ; la version choisie devra être figée dans le fichier de dépendances et le code devra suivre la documentation de cette ligne [R9]. Le dépôt ne doit pas mélanger des exemples v1 et v2.

10. Evidence Quality Gate (EQG)

10.1 Définition

L'EQG est un composant Python déterministe qui reçoit un snapshot d'observations normalisées et produit (a) des indicateurs par dimension, (b) une liste de problèmes détectés, (c) une décision de politique, et (d) une explication lisible. Il ne génère pas la cause racine et ne remplace pas le LLM ; il définit quelles affirmations diagnostiques le workflow peut autoriser.

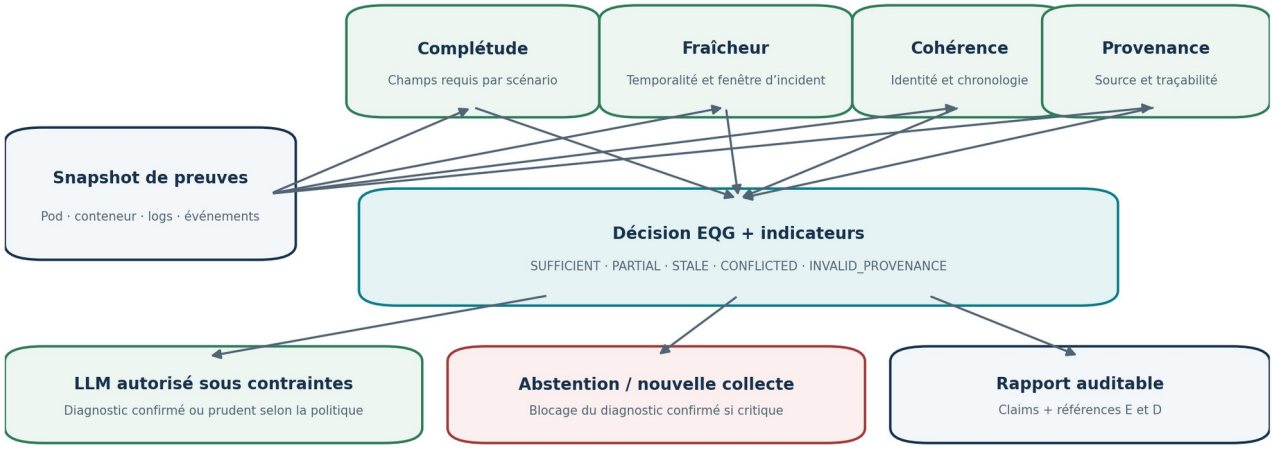


Figure 2 - Contrôles et décisions de l'Evidence Quality Gate.

10.2 Dimensions contrôlées

Dimension	Question	Exemples de règles
Complétude	Les champs indispensables au scénario sont-ils présents ?	Statut conteneur requis ; logs précédents attendus pour scénario crash si disponibles ; état de terminaison requis pour OOM.
Fraîcheur	Les preuves sont-elles pertinentes pour la fenêtre de l'incident ?	Collection après déclaration ou dans fenêtre définie ; preuve liée au Pod UID et au conteneur visé.
Cohérence	Les preuves décrivent-elles la même ressource et une chronologie compatible ?	Pod UID, container name/ID, restart count, timestamps ; contradictions non résolues.
Provenance	La source et le chemin de collecte sont-ils traçables ?	Source, nom d'outil, namespace, Pod UID, collected_at, snapshot_id, hash.

10.3 Schéma de preuve

```
{
  "evidence_id": "E-0001",
  "snapshot_id": "SNAP-INC-001",
  "incident_id": "INC-001",
  "source_type": "pod_status",
  "tool_name": "get_pod_status",
  "namespace": "opspilot-lab",
  "pod_name": "demo-api",
  "pod_uid": "<uid>",
  "container_name": "api",
  "container_id": "<container-id-if-available>",
  "collected_at": "2026-11-02T10:30:00Z",
  "observed_at": null,
  "resource_version": "<opaque-value>",
  "payload_hash": "<sha256>",
  "payload": {"restart_count": 3}
}
```

Exemple de contrat proposé. Les champs inconnus doivent rester null/unknown ; ne jamais les fabriquer.

10.4 Règles importantes de temporalité et d'identité

- resourceVersion identifie une version d'objet dans l'API Kubernetes ; ce n'est pas un timestamp. Il ne doit pas être comparé comme une horloge.

- La comparaison de resourceVersion n'est valide que dans les conditions décrites par l'API et pour le même type de ressource ; ne pas comparer directement Pod et Event [R12].
- Un log previous n'est pas automatiquement périmé : il peut décrire la dernière instance de conteneur qui a échoué et être la preuve la plus utile [R5].
- La freshness doit tenir compte de collected_at, de l'intervalle d'incident, du Pod UID, du container ID si disponible, du restartCount et du type de preuve.
- Les événements peuvent exposer eventTime, firstTimestamp, lastTimestamp, involvedObject et count ; conserver les champs effectivement disponibles [R13].
- Pod phase et container state doivent rester séparés ; la phase Running ne signifie pas que tous les conteneurs sont stables [R4].
- Les limites et requests mémoire appartiennent au spec du Pod/containeur ; reason et exitCode appartiennent au status/lastState.

10.5 Politique de décision

Décision	Condition indicative	Droit de génération
SUFFICIENT	Champs critiques présents ; provenance valide ; identité et temps cohérents ; aucun conflit critique.	Diagnostic confirmé autorisé, sous réserve de validation des claims.
PARTIAL	Certaines informations non critiques manquent ou un champ essentiel non bloquant est indisponible.	Hypothèses prudentes ; limites explicites ; pas de certitude injustifiée.
STALE	Preuve critique non rattachable à la fenêtre/instance de l'incident.	Pas de diagnostic confirmé ; demander une nouvelle collecte.
CONFLICTED	Observations critiques incompatibles pour la même identité et fenêtre, après règles de résolution.	Abstention sur la cause en conflit ; présenter le conflit.
INVALID_PROVENANCE	Source/identité/hash/collecte requis manquant ou invalide.	Preuve rejetée ; diagnostic confirmé bloqué si la preuve est critique.

Les dimensions sont conservées séparément sous forme de flags. Un snapshot peut être simultanément PARTIAL et STALE ; la décision principale est calculée par une politique de priorité documentée. Les seuils de fraîcheur seront configurables par source et scénario, sans TTL universel arbitraire.

10.6 Matrice de preuves requises par scénario

Scénario	Champs / observations attendus	Absence critique typique
CrashLoopBackOff	Pod UID, phase, container state Waiting reason, restartCount, lastState terminated, logs current/previous si disponibles, événements liés.	Pas d'état de conteneur ou identité Pod ; absence de logs n'autorise pas à inventer la cause.
ImagePullBackOff	Pod UID, image déclarée dans spec, waiting reason, événements Failed/ErrImagePull/ImagePullBackOff, timestamps.	Aucun événement/image de la ressource ; pas de logs applicatifs attendus si conteneur non démarré.
OOMKilled	Pod UID, lastState terminated reason, exitCode si présent, resource limits depuis spec, restartCount, événements/logs disponibles.	Absence d'état terminé ou de spec ressource ; exit code 137 ne doit pas être supposé.

10.7 Validation après génération

Le LLM produit un rapport structuré qui distingue claims, statut, références E et D, limites et actions de collecte recommandées. Un validateur Python indépendant vérifie que les IDs E/D existent, que les citations renvoient au bon type de source et que le niveau de certitude respecte la décision EQG. Si le gate bloque le diagnostic confirmé, le validateur rejette toute sortie qui présente une cause comme certaine. Un prompt seul n'est pas un mécanisme d'application de politique.

11. Orchestration LLM et génération du rapport

11.1 Workflow unique

START
-> Validate incident request
-> Collect read-only Kubernetes evidence (MCP)
-> Normalize and freeze snapshot
-> Evaluate evidence quality (EQG)
-> Retrieve version-compatible documentation (RAG)
-> Build constrained LLM context
-> Generate structured investigation report
-> Validate evidence IDs, document citations and permitted status
-> Persist report and audit trail
-> END

11.2 Rôle de LangChain et LangGraph

Composant	Usage	Exigence
LangChain	Documents, text splitters, embeddings, vector store/retrievers et composants de RAG.	Utilisé dans le pipeline documentaire.
LangGraph	Nœuds et état du workflow ; branchement selon EQG ; reprise sur erreurs si nécessaire.	Un seul graphe d'investigation ; pas de multi-agent obligatoire.
MCP client	Découverte et invocation du serveur de collecte.	Les outils doivent être réellement appelés et leurs résultats inspectables.

11.3 Modèle de rapport structuré

```
{
  "incident_id": "INC-001",
  "eqg": {
    "decision": "PARTIAL",
    "dimensions": {
      "completeness": "PARTIAL",
      "freshness": "PASS",
      "consistency": "PASS",
      "provenance": "PASS"
    },
    "issues": ["previous_logs_unavailable"]
  },
  "diagnosis": {
    "status": "tentative",
    "root_cause": null,
    "hypotheses": [
      {"statement": "Possible application startup configuration error",
       "evidence_ids": ["E-0002"],
       "document_ids": ["D-0004"],
       "limitations": ["Previous container logs unavailable"]}
    ]
  },
  "missing_information": ["previous_container_logs"],
  "recommended_collection": ["Retry get_pod_logs with previous=true"],
  "generated_at": "2026-11-02T10:35:00Z"
}
```

Exemple schématique : root_cause reste null si aucune cause confirmée n'est autorisée.

11.4 Gestion des erreurs

Erreur	Comportement attendu
MCP indisponible	Marquer la collecte en échec ; ne pas remplacer par des données inventées ; proposer retry limité.
Logs vides / non disponibles	Conserver status=unavailable et la raison ; l'EQG applique la règle scénario.
Version Kubernetes inconnue	Ne pas appliquer de filtre version implicite ; demander la version ou limiter la réponse.
Aucun passage RAG	Répondre que la documentation pertinente n'a pas été retrouvée ; citer zéro document.
Sortie LLM invalide	Rejeter le rapport, enregistrer erreur de parsing et permettre un retry borné.
Contradiction non résolue	Bloquer cause confirmée et exposer les observations contradictoires.
Dépassement timeout	Marquer étape en erreur, conserver les étapes déjà terminées et ne pas prétendre que l'analyse est complète.

11.5 Modèle et fournisseur LLM

Le modèle définitif sera choisi après un test de faisabilité matériel et fonctionnel. Critères : capacité à produire du JSON structuré, compatibilité avec l'appel d'outils ou son orchestration par le client, qualité de réponse en anglais/français si nécessaire, coût et conditions de licence. Le même modèle, le même prompt et les mêmes paramètres doivent être conservés entre les configurations comparées. Aucun fournisseur payant ne doit être une dépendance obligatoire du MVP.

12. Protocole expérimental et benchmarking

L'évaluation est organisée en trois pistes indépendantes mais complémentaires. Cette séparation évite de conclure qu'un diagnostic est fiable uniquement parce que le retrieval a trouvé un bon passage, ou qu'un bon diagnostic a nécessairement utilisé des citations correctes.

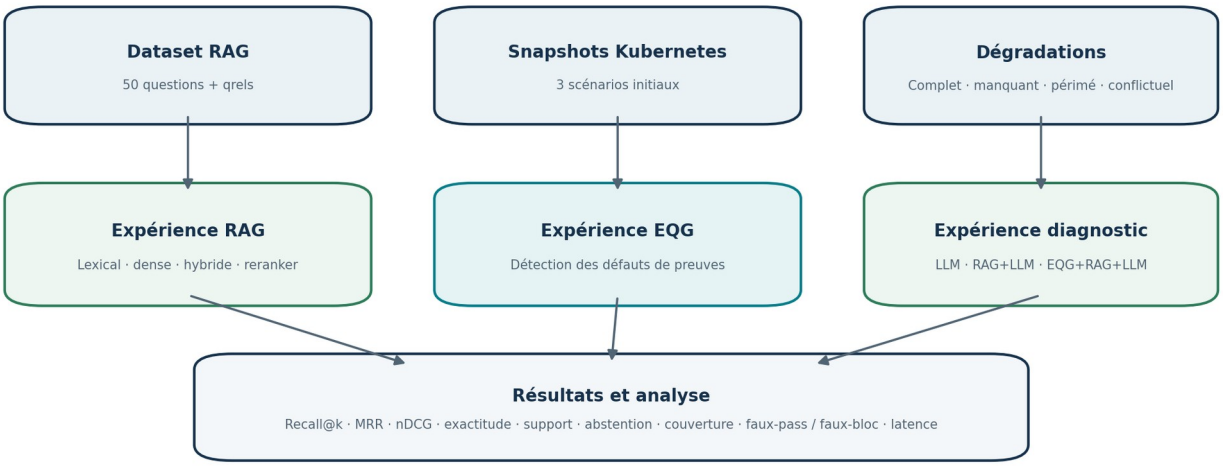


Figure 3 - Vue d'ensemble des trois pistes expérimentales.

12.1 Piste A - Benchmark RAG (exigence du cours)

Cette piste mesure la qualité de recherche documentaire et de génération. Elle constitue un benchmark autonome, indépendant des snapshots EQG.

Dataset RAG

Élément	Cible / règle
Questions	Au moins 50 questions documentaires couvrant Pod lifecycle, logs, images, ressources, probes, configuration, RBAC et événements.
Split initial	35 questions développement ; 15 questions holdout final.
Ground truth	Réponses de référence et liste de chunks pertinents (qrels) vérifiées manuellement.
Version	Chaque question précise version cible ou indique que la question n'est pas version sensible.
Anti-fuite	Séparer par concept/section ou famille de questions ; ne pas régler le système avec le holdout.
Révision	Les deux membres vérifient les labels et documentent les désaccords.

Configurations de retrieval

ID	Configuration	Variable étudiée
R1	Recherche lexicale PostgreSQL (tsvector + ts_rank / ts_rank_cd)	Baseline lexicale.
R2	Recherche dense (embeddings + pgvector)	Similarité sémantique.
R3	Hybride (lexical + dense, fusion documentée)	Complémentarité lexicale/sémantique.
R4	Hybride + cross-encoder reranker	Effet du reranking.

PostgreSQL ts_rank et ts_rank_cd sont des fonctions de ranking de recherche plein texte ; elles ne doivent pas être étiquetées BM25 [R22].

Métriques de retrieval

Métrique	Interprétation / formule
Recall@k	Nombre de passages pertinents retrouvés dans top-k / nombre total de passages pertinents connus.
Precision@k	Nombre de passages pertinents dans top-k / k.
MRR	Moyenne de 1 / rang du premier passage pertinent pour chaque requête.
nDCG@k	Gain cumulé actualisé normalisé ; mesure la qualité de l'ordre des résultats.
Latence retrieval	Durée entre requête et liste finale ; p50 et p95 si l'échantillon le permet.

Métriques de génération RAG

Métrique	Définition opérationnelle
Exactitude réponse	Réponse conforme à la réponse de référence et sans erreur factuelle importante.
Faithfulness / groundedness	Affirmations factuelles soutenues par les passages récupérés.
Exactitude de citation	Chaque citation D identifie un passage réel et soutient l'affirmation associée.
Complétude	Réponse couvre les éléments essentiels attendus, sans ajouter de détails non supportés.
Abstention documentaire	Système indique qu'il n'a pas de preuve documentaire suffisante lorsque le retrieval échoue.

Expérience version ON / OFF

Comparer récupération sans filtre de version et récupération avec filtre de version correct. L'analyse principale porte sur un sous-ensemble de questions réellement sensibles à la version. Les réponses et pages doivent être inspectées manuellement pour confirmer que les versions présentent une différence utile. Si aucun contenu ne diffère, la question ne peut pas démontrer l'avantage du filtrage.

12.2 Piste B - Benchmark de l'EQG

Cette piste mesure la capacité du gate à détecter les défauts définis dans les snapshots, avant toute génération. Le label attendu est défini au niveau de chaque dimension et du statut agrégé.

Condition	Construction	Label attendu
Complète	Toutes les observations requises pour le scénario sont présentes et associées à l'incident.	Qualité valide / décision potentielle SUFFICIENT.
Manquante	Retrait contrôlé d'un champ ou d'une observation déclarée critique/non critique.	PARTIAL ou blocage selon la criticité prévue.
Périmée	Preuve liée à une autre fenêtre ou instance ; identité temporelle incompatible.	STALE ou provenance invalide selon le défaut.
Contradictoire	Deux observations incompatibles pour la même identité et période, construites explicitement.	CONFLICTED.

Taille de l'évaluation EQG

MVP : 3 scénarios x 4 conditions = 12 cas de base. Extension : 6 scénarios x 4 conditions = 24 cas. Les quatre conditions d'un scénario dérivent d'un snapshot parent immuable. Les variantes ne sont pas des incidents indépendants ; elles sont des manipulations contrôlées d'un incident source. Les résultats seront décrits comme exploratoires et leur effectif exact sera toujours indiqué.

Métriques EQG

Métrique	Définition
EQG false-pass rate	Cas dégradés acceptés à tort comme SUFFICIENT / nombre de cas dégradés.
EQG false-block rate	Cas complets valides bloqués à tort / nombre de cas complets valides.
Détection par dimension	Précision, rappel et F1 pour les labels completeness, freshness, consistency et provenance.
Exactitude décision	Décision agrégée du gate conforme au label de référence.
Taux de non-résolution	Conflits qui restent non résolus et sont correctement signalés.

12.3 Piste C - Benchmark end-to-end du diagnostic

Configuration	Entrées	But
D1 - Evidence + LLM	Snapshot opérationnel sans documentation RAG et sans EQG.	Baseline de diagnostic fondé sur observations.
D2 - Evidence + RAG + LLM	Même snapshot + documentation récupérée ; gate désactivé.	Mesurer l'effet du contexte documentaire.
D3 - Evidence + RAG + EQG + LLM	Même snapshot et RAG ; gate actif et politique appliquée.	Mesurer l'effet incrémental de l'EQG.

Pour une comparaison équitable, conserver le même modèle, le même prompt de base, les mêmes snapshots, la même documentation et le même budget de contexte. La différence essentielle entre D2 et D3 est l'activation de l'EQG et de sa politique d'autorisation.

Métriques end-to-end

Métrique	Définition
Root Cause Accuracy	Cas avec cause racine correcte parmi les cas où une cause est présentée comme diagnostic ; toujours afficher le dénominateur.
Unsupported Diagnosis Rate	Diagnostics confirmés comportant au moins une affirmation de cause non soutenue par les preuves / diagnostics confirmés.
Evidence Support Precision	Affirmations factuelles liées à une preuve E valide et réellement supportive / toutes les affirmations factuelles évaluées.
Citation correctness	Citations documentaires exactes et pertinentes / citations documentaires évaluées.
Abstention precision / recall / F1	La classe positive est « devrait s'abstenir » selon la vérité terrain.
Diagnostic coverage	Nombre de cas avec diagnostic non abstenu / nombre total de cas.
Latency / cost	Durée par étape et totale ; tokens et coût monétaire si modèle payant.

12.4 Conditions de validité expérimentale

- Créer et geler un snapshot original avant de générer des variantes dégradées.
- Conserver un identifiant parent pour relier toutes les variantes au même incident.
- Ne pas utiliser les labels holdout pour régler prompts, seuils ou retrieval.
- Fixer versions de modèle, prompt, corpus, paramètres de génération et contexte comparé.
- Enregistrer appels de tools, erreurs et temps de chaque étape.
- Les répétitions d'un même cas mesurent la variabilité d'exécution ; elles n'augmentent pas le nombre d'incidents indépendants.
- Rapporter les effectifs par métrique et les résultats négatifs ; éviter toute généralisation au-delà du corpus.
- Faire annoter les cas par les deux étudiants et résoudre/documenter les désaccords.

13. Interface utilisateur et API

13.1 Écrans web du MVP

Écran	Fonctions essentielles	Données affichées
Tableau de bord	Résumé des incidents, derniers runs et statut du laboratoire.	Nombre d'incidents, statut cluster, dernières évaluations.
Espace incident	Sélection namespace/Pod, lancement de collecte et investigation.	Identité de ressource, statut collecte et étapes.
Preuves & EQG	Explorer les preuves E, les flags et la décision EQG.	Pod phase, états de conteneurs, logs, événements, provenance, erreurs.
Diagnostic	Lire les hypothèses, citations D, preuves E et limites.	Claims, niveau de diagnostic, abstention, recommandations de collecte.
Benchmark	Lancer / consulter les runs et comparer les configurations.	Recall@k, MRR, nDCG, exactitude, support, coverage et coûts.
Base documentaire	Voir le manifest et les versions du corpus.	Documents, sections, versions, statut d'ingestion, licence.

13.2 Principes UX

- Distinguer visuellement observation réelle, passage documentaire, hypothèse générée et conclusion validée.
- Afficher les appels MCP et les données collectées, sans exposer secrets ou informations sensibles.
- Afficher les raisons EQG et les champs manquants ; ne pas résumer la décision par un score opaque.
- Proposer un état vide, en chargement, succès partiel et erreur pour chaque écran.

- Ne pas présenter un pourcentage de confiance du LLM comme une probabilité calibrée sans étude spécifique.

13.3 API indicative

Méthode / route	Usage	Réponse principale
POST /api/incidents	Créer un incident de test.	incident_id, état initial.
GET /api/incidents/{id}	Lire incident et historique.	Incident + investigations.
POST /api/incidents/{id}/collect	Lancer les outils MCP et créer un snapshot.	snapshot_id, références E, erreurs.
POST /api/incidents/{id}/evaluate-evidence	Évaluer snapshot avec EQG.	decision, dimension results, issues.
POST /api/incidents/{id}/investigate	Lancer workflow RAG/LLM selon policy.	report_id, statut, claims.
GET /api/reports/{id}	Lire rapport structuré.	Diagnostics, refs E/D et limites.
GET /api/documents	Lister manifest documentaire.	Documents/version/status.
POST /api/evaluations/rag	Exécuter benchmark RAG.	evaluation_run_id, métriques.
POST /api/evaluations/eqg	Exécuter benchmark EQG.	evaluation_run_id, confusion matrix.
GET /api/evaluations/{id}	Consulter les métriques.	Run et résultats.

Les routes sont indicatives et seront figées lors de la conception API. Les appels MCP internes ne doivent pas être exposés directement au navigateur sans contrôle d'accès.

14. Modèle de données

14.1 Entités principales

Entité	Attributs clés	Rôle
User	user_id, role, created_at	Utilisateur et contrôle d'accès.
Incident	incident_id, namespace, target_kind, target_name, target_uid, started_at, status	Contexte d'investigation.
EvidenceSnapshot	snapshot_id, incident_id, captured_at, cluster_version, source_manifest	Ensemble figé d'observations.
EvidenceItem	evidence_id, snapshot_id, source_type, tool_name, identity, timestamps, payload_hash, payload	Observation atomique et traçable.
EQGResult	eqg_result_id, snapshot_id, decision, dimension_flags, issues, policy_version	Résultat de validation déterministe.
Document	document_id, source_url, version, license, content_hash, collected_at	Document source.
DocumentChunk	chunk_id, document_id, section, content, embedding, lexical_vector, metadata	Passage indexé.
Investigation	investigation_id, incident_id, snapshot_id, eqg_result_id, model_config, prompt_version, status	Exécution du workflow.
DiagnosisClaim	claim_id, investigation_id, statement, status, evidence_ids, document_ids	Affirmation du rapport.
EvaluationQuestion	question_id, split, version, expected_answer, relevant_chunk_ids	Question de benchmark RAG.
EvaluationRun	run_id, experiment_config, dataset_version, started_at, metrics	Exécution d'une expérience.
EvaluationCase	case_id, parent_snapshot_id, degradation_type, expected_eqg_label, expected_diagnosis	Cas EQG / diagnostic.

14.2 Principes de persistance

- Un snapshot original est immuable ; les variantes dégradées référencent son parent et décrivent la transformation.
- Les résultats EQG sont versionnés par version de règles afin de reproduire une évaluation.
- Les chunks documentaires conservent le lien vers le document, la section et la version.
- Chaque Investigation enregistre modèle, version de prompt, paramètres, corpus et snapshot.

- Les preuves brutes sensibles ne doivent pas être enregistrées sans nécessité ; le laboratoire ne contient que des données synthétiques/de test.

14.3 Indexation

Index PostgreSQL sur incident_id, snapshot_id, namespace, pod_uid, source_type, collected_at, version documentaire, document_id et split d'évaluation. L'index vectoriel pgvector sera créé après sélection du modèle d'embedding et validation du volume ; il n'est pas nécessaire pour un corpus lexical seul.

15. Sécurité, éthique et limites

15.1 Modèle de menace simplifié

Menace	Impact potentiel	Mesure de réduction
Prompt injection dans logs / documents	Le modèle traite des instructions malveillantes comme des commandes.	Données externes clairement délimitées ; aucun outil shell ; policy déterministe ; tests adversariaux.
Permission excessive Kubernetes	Lecture de données sensibles ou mutation de ressources.	RBAC least privilege ; aucun Secret ; tests can-i ; audit des droits.
Preuve issue d'un autre Pod	Diagnostic attribué à tort au workload courant.	UID, container identity, snapshot_id et fenêtre temporelle.
Sortie LLM non conforme	Cause confirmée malgré EQG en conflit / stale.	Schéma strict et validateur post-génération.
Citation incorrecte	Fausse impression de preuve.	Validation IDs, section, contenu et support claim-level.
Exposition secrets dans logs	Fuite de tokens ou données sensibles.	Application de test sans vrais secrets ; filtres/redaction ; no Secrets RBAC.
Dépendance externe / coût	Blocage du projet ou coût inattendu.	Mode local prioritaire ; provider facultatif ; quotas et estimation de coût.

15.2 Limites scientifiques de l'EQG

- Un gate déterministe ne peut pas garantir qu'un journal ou une observation système reflète la réalité ; il peut seulement vérifier les propriétés disponibles et les règles définies.
- Les règles de fraîcheur dépendent du type de preuve, de l'incident et des timestamps exposés par l'API.
- Certaines contradictions apparentes peuvent provenir d'états successifs ; leur résolution nécessite identité et chronologie.
- Un gate peut produire des faux-pass et faux-blocs ; ces erreurs sont des résultats à mesurer, pas des cas à masquer.
- Un petit nombre de scénarios locaux limite la généralisation à des clusters et incidents de production.
- La performance dépend du modèle, de la qualité du prompt, de la version de documentation et de l'implémentation du collector.

15.3 Politique de remédiation

Limite ferme du MVP

OpsPilot ne redémarre, ne supprime, ne modifie, ne scale et ne déploie aucun objet Kubernetes. Le projet se limite à l'observation, à l'explication et à la recommandation de collecte complémentaire. La création éventuelle d'un ticket interne OpsPilot est une extension séparée qui nécessiterait une confirmation humaine explicite.

15.4 Utilisation responsable des sources

Les documents Kubernetes seront attribués conformément à la licence du dépôt officiel. Les données de laboratoire seront identifiées comme incidents contrôlés et non comme incidents de production. Les résultats de benchmark seront rapportés avec leurs effectifs, versions et limites. Les citations de travaux existants ne seront pas assimilées à une reproduction ou à une validation indépendante de ces travaux.

16. Tests, qualité logicielle et pratiques MLOps

16.1 Stratégie de tests

Niveau	Exemples	Critère
Unitaires EQG	Champs requis absents, timestamps, UID incohérents, flags de dimension.	Règles déterministes et cas frontières validés.
Unitaires schema	Validation JSON des preuves, rapports, questions et snapshots.	Schémas valides et champs requis.
Intégration MCP	Appel tools/list et tools/call ; réponse cluster réelle.	Outils fonctionnels et erreurs explicites.
Sécurité RBAC	Can-i get/list pods, get pods/log, get/list events ; refus secrets/update/delete.	Aucun privilège non prévu.
RAG	Ingestion, filtrage version, retrieval, citation et update.	Métriques calculées sur qrels.
Workflow	EQG partial/stale/conflicted, panne MCP, sortie LLM invalide.	Transitions et policy correctes.
End-to-end	Incident depuis l'interface jusqu'au rapport.	Références E/D vérifiables.
Non-régression	Jeu de cas gelé rejoué après modification.	Écarts documentés.

16.2 CI/CD et dépôt Git

- Conserver un historique Git progressif avec commits atomiques et messages explicites.
- Utiliser feature branches et pull requests ; chaque étudiant révise les contributions de l'autre.
- CI : lint, type checks ciblés, tests unitaires, validation JSON/YAML et tests d'intégration non secrets.
- Figer les dépendances avec un lockfile ou contraintes versionnées et documenter la version Python.
- Fournir .env.example sans secrets ; ignorer .env, tokens, kubeconfig et logs sensibles.
- Versionner les manifests, corpus manifest, règles EQG, prompts et protocoles d'évaluation.
- Les runs de benchmark doivent enregistrer la configuration et produire des artefacts de résultats reproductibles.

16.3 MLOps / LLMOps

MLflow peut être utilisé pour suivre les expériences et paramètres. Un outil de tracing tel que Langfuse peut être ajouté si l'installation locale et la charge le permettent. Ces outils sont des choix de soutien, non des exigences pédagogiques obligatoires. La priorité reste l'enregistrement local des runs, prompts, paramètres, timings, tokens et scores dans un format réutilisable.

16.4 Critères de qualité du code

- Type hints sur les interfaces et modèles de données.
- Séparation entre règles EQG, appels réseau, persistance et génération.
- Timeouts et traitement des exceptions des appels Kubernetes, MCP, base et modèle.
- Aucun appel réseau implicite dans les tests unitaires ; fixtures pour cas déterministes.
- Chaque règle EQG possède un identifiant, une justification, un test positif et un test négatif.

17. Organisation de l'équipe et planning

17.1 Membres

Membre	Responsabilité principale	Livrables individuels identifiables
Bilel Amri	RAG, gestion documentaire/version, schémas de preuves, EQG, intégration LLM/LangGraph, protocole et évaluation.	Corpus manifest, ingestion/retrieval, règles EQG, tests EQG, rapport de benchmark RAG et analyse expérimentale.
Mohamed Amine Gotai	Laboratoire Kubernetes, scénarios, serveur MCP/RBAC, frontend, intégration API et CI.	Cluster/configs, scénarios reproductibles, outils MCP, tests RBAC, interface, workflow CI.

Membre	Responsabilité principale	Livrables individuels identifiables
Les deux	Architecture commune, intégration bout-en-bout, annotations, revue croisée, rapport et présentation.	PR revues, jeux de tests validés, rapport commun, démonstration maîtrisée.

Validation d'équipe

Cette répartition est une proposition. Les deux étudiants doivent confirmer leur accord. Les responsabilités primaires n'excluent pas la compréhension partagée de l'architecture et la revue mutuelle du code.

17.2 Planning indicatif sur huit semaines

Semaine	Objectif	Tâches principales	Sortie / jalon
S0 / préparation	Valider le périmètre et l'environnement.	Revue de littérature ciblée ; accord enseignant ; vérifier Docker, Kind, RAM, disk, versions.	GO/NO-GO laboratoire.
S1	Laboratoire Kubernetes.	Cluster Kind, application FastAPI, SC-01 CrashLoopBackOff ; collecte manuelle status/logs/events.	Premier snapshot réel validé.
S2	Étendre le laboratoire et MCP.	SC-02, SC-03 ; 4 outils MCP ; RBAC minimal ; snapshots figés.	Collecte MCP réelle et testée.
S3	Evidence schema et EQG.	Normalisation, provenance, complétude, fraîcheur, cohérence ; jeux positifs/négatifs.	EQG unitaire et explicable.
S4	Pipeline RAG.	Corpus 1.36/1.37, metadata, index, lexical/dense/hybride.	Ingestion et retrieval vérifiés.
S5	Workflow AI et interface.	LangChain, LangGraph, modèle, rapports structurés, validator, interface de base.	Parcours d'incident complet.
S6	Benchmarking.	50 questions, qrels, holdout ; cas EQG dégradés ; runs D1/D2/D3.	Résultats préliminaires.
S7	Finalisation.	Tests sécurité, correction, analyse erreur, rapport, README, vidéo et soutenance.	Livrables remis et démonstration.

17.3 Jalons GO / NO-GO

Jalon	Condition de passage	Action si échec
G0 - Environnement	Docker engine et Kind fonctionnels.	Réduire la charge locale ou utiliser un environnement alternative autorisé.
G1 - Incident réel	Un scénario génère preuves réelles et reproductibles.	Corriger le scénario/collector avant EQG.
G2 - MCP sécurisé	Outils répondent et RBAC refuse les mutations/secrets.	Bloquer l'intégration LLM tant que RBAC n'est pas vérifié.
G3 - EQG testable	Règles deterministic testées sur cas complets et dégradés.	Clarifier les règles et vérité terrain.
G4 - RAG mesuré	Corpus, qrels, retrieval et citations fonctionnels.	Réduire le corpus, corriger métadonnées/chunking.
G5 - Démonstration	Parcours complet reproductible depuis UI.	Prioriser stabilité et limites honnêtes au détriment des extensions.

18. Livrables et critères d'acceptation

18.1 Livrables pédagogiques

Livrable	Contenu attendu
D1 - Dépôt GitHub	Code documenté, README, architecture, instructions, tests, CI et historique de commits progressif.

Livrable	Contenu attendu
D2 - Rapport technique	Problématique, état de l'art, conception, données, EQG, RAG, MCP, sécurité, méthodologie, résultats, limites.
D3 - Démonstration	Incident contrôlé, collecte réelle, décision EQG, retrieval documentaire et rapport avec preuves.
D4 - Présentation	Objectif métier, question de recherche, architecture, expérimentation, résultats et limites.

18.2 Critères d'acceptation MVP

ID	Critère	Preuve d'acceptation
AC-01	Le cluster Kind et l'application de démonstration démarrent selon les instructions.	Commande reproductible + sortie vérifiée.
AC-02	Au moins trois scénarios initiaux sont reproductibles.	Manifest + expected observations + snapshot réel.
AC-03	Les quatre outils MCP sont invoqués par le client et retournent des données réelles.	Test MCP / preuve de démonstration.
AC-04	Le ServiceAccount MCP ne peut pas modifier les Pods ni lire Secrets.	Tests kubectl auth can-i et manifest RBAC.
AC-05	Les observations sont normalisées avec identité, timestamps, source et snapshot.	Validation schéma + tests provenance.
AC-06	EQG détecte les cas contrôlés de preuve complète, manquante, périmée et contradictoire.	Matrice de confusion et exemples.
AC-07	RAG ingère corpus versionné et retourne des passages citables.	Manifest + tests metadata et citation.
AC-08	Benchmark RAG comporte au moins 50 questions et des qrels vérifiés.	Dataset + rapport Recall@k/MRR/nDCG.
AC-09	Le rapport sépare références E et D et respecte la décision EQG.	Tests post-validation.
AC-10	L'interface web permet de consulter incident, preuves, décision et résultats.	Démo end-to-end.
AC-11	Le rapport final présente résultats et limites sans prétendre à une supériorité non démontrée.	Revue du rapport et des tableaux.

18.3 Exigences du cours : traçabilité

Exigence du cours	Réponse OpsPilot	Module cible	Test / démonstration
Gestion documentaire : RAG	Corpus versionné, métadonnées, mise à jour, citations.	backend/app/rag/	Ingestion, metadata, citations.
Tool Calling / MCP	Serveur MCP avec outils Kubernetes réellement exécutés.	mcp_server/	MCP client integration + RBAC.
LangChain et/ou LangGraph	LangChain pour RAG ; LangGraph pour workflow.	backend/app/rag/ ; agents/	Tests retrieval et workflow.
Benchmark retrieval	4 configurations et qrels.	evaluation/rag/retrieval/	Recall@k, MRR, nDCG.
Benchmark génération	Réponses de référence, groundedness et citations.	evaluation/rag/generation/	Correctness, faithfulness, citation.
Interface web/mobile	React + TypeScript.	frontend/	Test parcours de bout en bout.
Livrables	GitHub, rapport, démonstration et présentation.	docs/, README, release artifacts	Historique Git et soutenance.

Le cahier des charges de l'enseignant joint au cours indique également un objectif d'au moins 50 questions, Recall@k, MRR et une expérimentation de fine-tuning. L'email fourni indique explicitement que le fine-tuning est facultatif. La présente proposition retient 50 questions comme cible de conformité au document joint et laisse le fine-tuning hors MVP, à confirmer auprès de l'enseignant [R23].

19. Analyse des risques

ID	Risque	Prob.	Impact	Mitigation / plan de secours
R-01	Docker Engine non disponible ou ressources insuffisantes.	Moy.	Fort	Vérifier matériel avant Kind ; réduire les composants ; exécuter services séquentiellement.
R-02	Scénarios non reproductibles.	Moy.	Fort	Manifests versionnés, seeds et procédures reset ; un scénario validé avant extension.
R-03	Outil MCP trop permissif.	Faible/Moy.	Critique	RBAC minimal, tests can-i, aucun outil write et revue croisée.
R-04	Preuves sans identité ou horodatage fiable.	Moy.	Fort	Schéma obligatoire ; champs unknown explicites ; provenance testée.
R-05	EQG rejette trop de cas valides.	Moy.	Fort	Mesurer false-block, adapter règles par scénario, ne pas utiliser de TTL universel.
R-06	EQG laisse passer des cas dégradés.	Moy.	Fort	Mesurer false-pass, tests adversariaux et validation post-génération.
R-07	Corpus documentaire incohérent entre versions.	Moy.	Moy.	Manifest figé, filtrage version, validation des questions sensibles.
R-08	Modèle ne respecte pas le JSON/tool workflow.	Moy.	Moy.	Test de modèle précoce, modèle alternatif, validation stricte.
R-09	50 questions insuffisamment annotées.	Moy.	Fort	Double annotation par l'équipe, adjudication, qrels, split gelé.
R-10	Scope trop ambitieux pour 8 semaines.	Fort	Fort	Trois scénarios initiaux, pas de fine-tuning, pas de remédiation, extensions conditionnelles.
R-11	Postmortem externe non redistribuable.	Moy.	Moy.	N'utiliser que les liens ou sources dont la licence est vérifiée ; corpus RAG basé d'abord sur docs officielles.
R-12	Fuite de données sensibles via logs ou provider LLM.	Faible/Moy.	Fort	Lab synthétique, redaction, local-first, secrets hors prompts et dépôt.

19.1 Risques scientifiques

- Les résultats peuvent montrer que l'EQG n'améliore pas les métriques ou qu'il réduit trop la couverture ; cela doit être rapporté honnêtement.
- Les dégradations synthétiques peuvent être trop simples ; décrire les règles de génération et utiliser des exemples réalistes, sans prétendre refléter tous les incidents de production.
- Le corpus de trois scénarios est petit ; éviter l'extrapolation aux clusters hétérogènes.
- Le LLM peut varier entre exécutions ; figer les paramètres et enregistrer le modèle/configuration.

20. Faisabilité et ressources

20.1 Ressources logicielles

Composant	Choix proposé	Statut / remarque
Langage backend	Python >= 3.11	Le SDK MCP Python v2 requiert Python 3.11+ [R9].
API	FastAPI	Dépendances et versions figées après test.
MCP	SDK Python officiel MCP v2	Vérifier API v2 et figer la version choisie.
RAG	LangChain	Document loaders, splitters, embeddings, retrievers [R24].
Workflow	LangGraph	Un graphe, état explicite et checkpoint optionnel [R25].
Base	PostgreSQL	Stockage relationnel, plein texte.
Recherche dense	pgvector	Optionnelle selon test d'embeddings.
Cluster	Kind	Cluster local en conteneurs Docker [R8].
Frontend	React + TypeScript	MVP web responsive.
Tests	pytest, tests API/MCP, validation YAML/JSON	Exécutions locales et CI.

20.2 Modèle LLM et budget

Le modèle ne sera pas fixé dans le présent document avant le test matériel et de compatibilité. Le choix se fera entre une option locale légère et, si autorisée, une API externe avec coût plafonné. Le provider retenu devra supporter les sorties structurées ou être compatible avec un mécanisme de parsing/retry. Aucun coût payant ne doit empêcher la démonstration du cœur du système.

20.3 Versions Kubernetes documentaires

À la date du 2 novembre 2026, les branches Kubernetes 1.37, 1.36 et 1.35 sont maintenues ; les releases publiées indiquent 1.37.1 et 1.36.5 comme derniers correctifs respectifs à la date de référence [R1]. Le corpus retient 1.36 et 1.37. La version du cluster de laboratoire sera choisie parmi les versions compatibles et figée dans les configurations ; l'étude de filtrage documentaire ne suppose pas nécessairement deux clusters actifs simultanément.

20.4 Coût et dépendances externes

- Logiciels du cœur : privilégier des outils open source et une exécution locale.
- Dépendance LLM : mode local ou provider optionnel ; coût/tokens consignés dans chaque run.
- Données : documentation Kubernetes officielle ; incidents de laboratoire générés par l'équipe.
- R2Act, Cloud-OpsBench et autres benchmarks : références bibliographiques, non dépendances du MVP.
- Aucune infrastructure cloud payante nécessaire pour le laboratoire initial.

20.5 Critère de faisabilité initial

GO / NO-GO

Le développement de l'EQG et du RAG débute après la réussite d'un incident réel dans Kind, avec collecte vérifiable de Pod/container status, logs et événements. Si le matériel ne permet pas la co-exécution de tous les services, le projet fonctionne en mode séquentiel et réduit.

21. Glossaire

Terme	Définition
AI Engineering	Conception, implémentation, évaluation et exploitation d'applications intégrant des modèles d'IA.

Terme	Définition
RAG	Retrieval-Augmented Generation : récupération de contenu externe avant génération.
MCP	Model Context Protocol : protocole permettant à un client de découvrir et invoquer des outils exposés par un serveur.
LangChain	Framework et abstractions pour documents, modèles, embeddings, retrievers et composants d'applications LLM.
LangGraph	Framework d'orchestration de workflows LLM avec état et transitions explicites.
Pod phase	Résumé de haut niveau du cycle de vie d'un Pod.
Container state	État d'un conteneur : Waiting, Running ou Terminated.
CrashLoopBackOff	Indication de back-off pendant des redémarrages répétés d'un conteneur ; pas une Pod phase.
Snapshot	Capture figée et identifiée des observations collectées pour un incident.
EQG	Evidence Quality Gate : validation déterministe de propriétés des preuves.
Provenance	Information permettant de retracer la source et le contexte d'une observation.
Holdout	Sous-ensemble de test non utilisé pour ajuster le système.
qrels	Jugements de pertinence reliant chaque question aux passages documentaires pertinents.
Abstention	Décision de ne pas présenter un diagnostic confirmé lorsque l'information est insuffisante.
Diagnostic coverage	Part des cas pour lesquels le système fournit un diagnostic non abstenu.
False-pass EQG	Cas dégradé que le gate accepte à tort comme suffisant.
False-block EQG	Cas complet valide que le gate bloque à tort.

22. Références

Les références ci-dessous servent à justifier les choix techniques et l'état de l'art au moment de la rédaction (2 novembre 2026). Les versions, licences et disponibilités peuvent évoluer ; elles seront revérifiées au moment du gel des dépendances et du corpus.

[R1] Kubernetes - Releases and support branches
https://kubernetes.io/releases/
Versions 1.37, 1.36 et 1.35 maintenues à la date de référence.

[R2] Kubernetes - Supported documentation versions
https://kubernetes.io/docs/home/supported-doc-versions/
Disponibilité des versions documentaires.

[R3] Kubernetes website repository
https://github.com/kubernetes/website
Dépôt officiel de la documentation ; licence CC-BY-4.0 affichée par le dépôt.

[R4] Kubernetes - Pod lifecycle
https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/
Pod phase, états de conteneur et CrashLoopBackOff.

[R5] Kubernetes - Debug Running Pods
https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/
Consultation des logs courants et précédents.

[R6] Kubernetes - Resource management for Pods and Containers
https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
Requests et limits de ressources.

[R7] Kubernetes website - licence du dépôt
https://github.com/kubernetes/website
Attribution et conditions de réutilisation de la documentation.

[R8] Kind - Kubernetes in Docker
https://kind.sigs.k8s.io/
Clusters Kubernetes locaux sur des nœuds conteneurisés.

[R9] MCP Python SDK - documentation v2

https://py.sdk.modelcontextprotocol.io/
Ligne stable v2, exigences Python et migration v1/v2.

[R10] Model Context Protocol - Tools specification
https://modelcontextprotocol.io/specification/2026-07-28/server/tools
Découverte, invocation, schémas et sécurité des outils.

[R11] Kubernetes - RBAC authorization
https://kubernetes.io/docs/reference/access-authn-authz/rbac/
Règles et sous-ressource pods/log.

[R12] Kubernetes - API concepts / resourceVersion
https://kubernetes.io/docs/reference/using-api/api-concepts/
Sémantique resourceVersion et limites de comparaison.

[R13] Kubernetes - Event API reference
https://kubernetes.io/docs/reference/kubernetes-api/cluster-resources/event-v1/
Event fields et références d'objet.

[R14] K8sGPT - MCP documentation
https://github.com/k8sgpt-ai/k8sgpt/blob/main/MCP.md
Exemple existant de diagnostic Kubernetes via MCP.

[R15] HolmesGPT - Kubernetes toolsets
https://holmesgpt.dev/dev/data-sources/builtin-toolsets/kubernetes/
Intégrations Kubernetes en lecture seule et MCP.

[R16] Botkube - project documentation
https://docs.botkube.io/
Outils Kubernetes et assistant d'observabilité.

[R17] ARGUS: MCP-Grounded Root Cause Analysis for Kubernetes Incidents
https://arxiv.org/abs/2608.23084
Travail de recherche sur RCA Kubernetes et observabilité via MCP.

[R18] Cloud-OpsBench: A Reproducible Benchmark for Agentic RCA in Cloud Systems
https://arxiv.org/abs/2603.00468
Benchmark de RCA reproductible ; repository : https://github.com/LLM4Ops/Cloud-OpsBench

[R19] TriFleetRCA: On-Premise LLM Root Cause Analysis for Kubernetes
https://arxiv.org/abs/2609.23766
RCA, retrieval, runbook guard et citation.

[R20] R2Act: Can LLMs Really Recover Microservice Failures?
https://arxiv.org/abs/2607.04623
Évaluation de validité d'action après diagnostic.

[R21] K8sGPT Calibration & Abstention Benchmark
https://github.com/Mayank-013/k8sGPT
Benchmark communautaire pour calibration / abstention.

[R22] PostgreSQL - Text Search Functions and Operators
https://www.postgresql.org/docs/current/functions-textsearch.html
ts_rank et ts_rank_cd ; ne pas les nommer BM25.

[R23] Document pédagogique AI Engineering
AI_Engineering_Project_Topics.pdf, fourni par l'étudiant
Projets proposés et attentes communes : RAG, Recall@k, MRR, MCP, 50 questions et fine-tuning mentionné.

[R24] LangChain - Component architecture
https://docs.langchain.com/oss/python/langchain/component-architecture
Document loaders, text splitters, embeddings, vector stores et retrieval.

[R25] LangGraph - Persistence and interrupts
https://docs.langchain.com/oss/python/langgraph/persistence
Checkpoints, état et workflows avec interruption / reprise.

Annexe A - Questions à confirmer avec l'enseignant

- Le ciblage de 50 questions documentaires prévu par le cahier pédagogique joint est-il attendu comme seuil obligatoire pour ce groupe ?

- La mention de fine-tuning dans le PDF commun est-elle facultative, compte tenu de l'email qui indique explicitement qu'il ne l'est pas ?
- Le sujet OpsPilot et son nouveau focus EQG sont-ils validés dans le cadre du cours AI Engineering ?
- La démonstration peut-elle être réalisée sur un cluster Kind local avec des incidents de laboratoire reproductibles ?
- Le protocole MCP attendu peut-il être implémenté avec le SDK Python v2 actuel ?
- Le planning réel et la date de soutenance permettent-ils l'extension à six scénarios, ou faut-il se limiter au MVP de trois scénarios ?

Annexe B - Décision de version

Cette V1.3 consolide les exigences pédagogiques, le projet EQG, le benchmark RAG, le protocole d'incident, la sécurité et la répartition de l'équipe. Toute modification ultérieure doit être enregistrée dans l'historique Git et dans le journal de décisions. La présente version doit être validée par l'enseignant avant d'être considérée comme périmètre contractuel du groupe.

Fin du cahier des charges

OpsPilot vise un prototype limité, reproductible et mesurable. La réussite du projet dépend de la qualité des preuves, des tests et de la transparence des résultats, et non du nombre de frameworks ou d'agents utilisés.